

Liskov Substitution Principle in C# with Examples

In this article, I will discuss the **Liskov Substitution Principle in C#** with Examples. Please read our previous article before proceeding to this article, where we discussed the [Open-Closed Principle in C#](#) with Examples. The Letter **L** in SOLID stands for the **Liskov Substitution Principle**, also known as LSP. As part of this article, we will discuss the following pointers in detail.

1. **What is the Liskov Substitution Principle in C#?**
2. **Example: Without using the Liskov Substitution Principle in C#**
3. **Example: Using the Liskov Substitution Principle in C#**
4. **Advantages and Disadvantages of the Liskov Substitution Principle in C#.**
5. **How to Use the Liskov Substitution Principle in C#?**

What is the Liskov Substitution Principle in C#?

The Liskov Substitution Principle (LSP), another SOLID principle, was introduced by Barbara Liskov. It states that objects of a superclass should be replaceable with objects of its subclasses without affecting the correctness of the program. That means a subclass should completely adhere to the behavior expected by the superclass. The Liskov Substitution Principle encourages a design where subclasses are substitutable for their base classes.

In simple words, we can say that when we have Parent-Child relationships, i.e., Inheritance Relationships between two classes, then if we successfully replace the object/instance of a parent class with an object/instance of the child class without affecting the behavior of the base class instance, it is said to be in Liskov Substitution Principle. If you are not getting this point properly, don't worry; we will see some real-time examples to understand this concept.

For example, a father is a teacher, whereas his son is a doctor. So here, in this case, the son can't simply replace his father even though both belong to the same family.

Example Without using the Liskov Substitution Principle in C#:

Let us first understand one example without using the Liskov Substitution Principle in C#. We will see the problem if we are not following the Liskov Substitution Principle, and then we will see how we can overcome such problems using the Liskov Substitution Principle. In the following example, first, we create the Apple class with the method GetColor. Then, we create the Orange class, which inherits the Apple class and overrides the GetColor method of the Apple class. The point is that an Orange cannot be replaced by an Apple, which results in printing the color of the apple as Orange, as shown in the example below.

```
using System;
namespace SOLID_PRINCIPLES.LSP
{
    class Program
    {
        static void Main(string[] args)
        {
            Apple apple = new Orange();
            Console.WriteLine(apple.GetColor());
        }
    }
}
```

```

    }
    public class Apple
    {
        public virtual string GetColor()
        {
            return "Red";
        }
    }
    public class Orange : Apple
    {
        public override string GetColor()
        {
            return "Orange";
        }
    }
}

```

As you can see in the above example, Apple is the base class, and Orange is the child class, i.e., there is a Parent-Child relationship. So, we can store the child class object in the Parent class Reference variable, i.e., **Apple apple = new Orange();** and when we call the GetColor, i.e., apple.GetColor(), then we are getting the color Orange, not the color of an Apple. That means the behavior changes once the child object is replaced, i.e., Apple stores the Orange object. This is against the LSP Principle. The Liskov Substitution Principle states that even if the child object is replaced with the parent, the behavior should not be changed. So, in this case, if we are getting the color Apple instead of Orange, it follows the Liskov Substitution Principle. That means there is some issue with our software design. Let us see how to overcome the design issue and make the application follow the Liskov Substitution Principle using C# Language.

Example Using the Liskov Substitution Principle in C#

Let's modify the previous example to follow the Liskov Substitution Principle using C# Language. First, we need a generic base Interface, i.e., IFruit, which will be the base class for both Apple and Orange. Now, you can replace the IFruit variable can be replaced with its subtypes, either Apple or Orage, and it will behave correctly. In the code below, we created the super IFruit as an interface with the GetColor method. Then, the Apple and Orange classes were inherited from the Fruit class and implemented the GetColor method.

```

using System;
namespace SOLID_PRINCIPLES.LSP
{
    class Program
    {
        static void Main(string[] args)
        {
            IFruit fruit = new Orange();
            Console.WriteLine($"Color of Orange: {fruit.GetColor()}");
            fruit = new Apple();
            Console.WriteLine($"Color of Apple: {fruit.GetColor()}");
            Console.ReadKey();
        }
    }
    public interface IFruit
    {

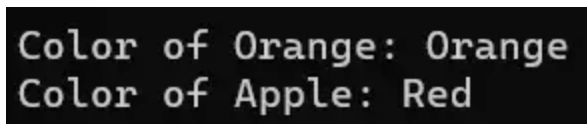
```

```

        string GetColor();
    }
    public class Apple : IFruit
    {
        public string GetColor()
        {
            return "Red";
        }
    }
    public class Orange : IFruit
    {
        public string GetColor()
        {
            return "Orange";
        }
    }
}

```

Now, run the application, and it should give the expected output, as shown in the image below. Here, we follow the LSP as we can change the object with its subtype without affecting the behavior.



```

Color of Orange: Orange
Color of Apple: Red

```

So, now Fruit can be any type and any color, but orange cannot be the color red. An apple cannot be of the color orange, meaning we cannot replace an orange with an apple, but fruit can be replaced with an orange or an apple because they are both Fruits; an apple is not an orange, and an orange is not an apple.

Note: You need to remember that, as we have the Inheritance concept, that does not mean we can randomly create the relationship between classes. We will not get any error or exception, but the behavior that we expect might not be. So, always make sure that the relationship and functionality we provide make sense. If it makes sense, then go for the inheritance relationship; if not, then don't go for the inheritance relationship.

Real-Time Example of Liskov Substitution Principle in C#: Bank Accounts

Let's see another real-time example of Bank Accounts to understand the Liskov Substitution Principle (LSP). Imagine you're building a banking system. The following example code is self-explained, so please go through the comment lines for a better understanding.

```

using System;
namespace LSPDemo
{
    //Imagine you have a base class BankAccount
    public class BankAccount
    {
        public string AccountNumber { get; set; }
        public decimal Balance { get; set; }
        public BankAccount(string accountNumber, decimal balance)
        {

```

```

        AccountNumber = accountNumber;
        Balance = balance;
    }
    public virtual void Deposit(decimal amount)
    {
        Balance += amount;
        Console.WriteLine($"Deposit: {amount}, Total Amount: {Balance}");
    }
    public virtual void Withdraw(decimal amount)
    {
        if (amount <= Balance)
        {
            Balance -= amount;
        }
        else
        {
            Console.WriteLine("Insufficient balance.");
        }
    }
}
//We have two derived classes: SavingsAccount and CurrentAccount
public class SavingsAccount : BankAccount
{
    public decimal InterestRate { get; set; }
    public SavingsAccount(string accountNumber, decimal balance, decimal
interestRate)
        : base(accountNumber, balance)
    {
        InterestRate = interestRate;
    }
    public override void Withdraw(decimal amount)
    {
        if (amount <= Balance)
        {
            Balance -= amount;
            Console.WriteLine($"AccountNumber: {AccountNumber}, Withdraw:
Balance: {Balance}");
        }
        else
        {
            Console.WriteLine($"AccountNumber: {AccountNumber}, Withdraw:
Insufficient Funds, Available Funds: {Balance}");
        }
    }
}
public class CurrentAccount : BankAccount
{
    public decimal OverdraftLimit { get; set; }
    public CurrentAccount(string accountNumber, decimal balance, decimal
overdraftLimit)
        : base(accountNumber, balance)

```

```

    {
        OverdraftLimit = overdraftLimit;
    }
    public override void Withdraw(decimal amount)
    {
        if (amount <= Balance + OverdraftLimit)
        {
            Balance -= amount;
            Console.WriteLine($"AccountNumber: {AccountNumber}, Withdraw:
Balance: {Balance}");
        }
        else
        {
            Console.WriteLine($"AccountNumber: {AccountNumber}, Exceeded (
Limit.");
        }
    }
}
//Testing the Liskov Substitution Principle
public class Program
{
    public static void Main()
    {
        BankAccount savingsAccount = new SavingsAccount("SA123", 1000m, 0.
        BankAccount currentAccount = new CurrentAccount("CA456", 1500m, 50
        Console.WriteLine("Before Transactions:");
        PrintAccountDetails(savingsAccount);
        PrintAccountDetails(currentAccount);
        savingsAccount.Withdraw(500m);
        currentAccount.Withdraw(2000m);
        Console.WriteLine("\nAfter Transactions:");
        PrintAccountDetails(savingsAccount);
        PrintAccountDetails(currentAccount);
        Console.ReadKey();
    }
    static void PrintAccountDetails(BankAccount account)
    {
        Console.WriteLine($"Account Number: {account.AccountNumber}, Balan
{account.Balance}");
    }
}
}

```

In this example:

We have a base class called BankAccount with Deposit and Withdraw methods. Two derived classes, SavingsAccount and CurrentAccount, inherit from it. Each derived class overrides the Withdraw method to implement account-specific withdrawal rules.

In the Main method, we create objects for SavingsAccount and CurrentAccount but define them as BankAccount types. This follows the Liskov Substitution Principle (LSP), which allows us to replace instances of derived classes with the base class without affecting the program's correctness.

When performing withdrawal transactions on these accounts, their types do not need to be specified. The Withdraw method for each account type is implemented correctly based on its actual type. So, when you run the above code, it will give you the following output.

```
Before Transactions:
Account Number: SA123, Balance: 1000
Account Number: CA456, Balance: 1500
AccountNumber: SA123, Withdraw: 500, Balance: 500
AccountNumber: CA456, Withdraw: 2000, Balance: -500

After Transactions:
Account Number: SA123, Balance: 500
Account Number: CA456, Balance: -500
```

This example illustrates how the Liskov Substitution Principle (LSP) enables us to operate with objects of derived classes (SavingsAccount and CurrentAccount) using the base class (BankAccount) interface without changing or altering the behavior of SavingsAccount and CurrentAccount type.

In the next article, I will discuss [Multiple Real-Time Examples of the Liskov Substitution Principle \(LSP\) in C#](#). In this article, I explain the Liskov Substitution Principle in C# with Examples. I hope you enjoy this Liskov Substitution Principle article.